Task 1: Generate an Encryption Key the Wrong Way

1.

```
21ky11@LINUX3:~/468/a1$ gcc task1.c -o task1
21ky11@LINUX3:~/468/a1$ ./task1
1768847605
54d49088ee86cbc05dc8602391f0845d
```

We use *the number of seconds elapsed since January 1, 1970, at 00:00 UTC* as the seed using srand().
Then we generate a pseudorandom number in the range of 0 to 2147483647 (2^{31}-1) then modulo it to get a number in the range of 0 to 255 (an 8-bit number, or one byte).
We generate 16 bytes (defined as the key size) and print them (each byte = 8 bits = 2 hex digits), a total of 32 hex digits printed as the second line of the output.

2.

```
21ky11@LINUX3:~/468/a1$ gcc task1.c -o task1
21ky11@LINUX3:~/468/a1$ ./task1
1768848369
67c6697351ff4aec29cdbaabf2fbe346
21ky11@LINUX3:~/468/a1$ ./task1
1768848371
67c6697351ff4aec29cdbaabf2fbe346
```

Now we can see that the same sequence of key bits is printed out, even at different timestamps.

3.

Why srand() is used:

- rand() is not random on its own without a seed!
- rand() is a pseudo-random number generator that is deterministic
- With srand() setting different seeds, it allows us to see different keys output

Why time() is used

- time() is a changing value
- Using time(NULL) (current timestamp) ensures:
    - Different program execution → different seed (unless you run quickly enough)
    - Different pseudo-random sequences output

Task 2: Guess the Key

1.

```
seed@cisc468:~/a1$ gcc task2.c -o task2 -lssl -lcrypto
seed@cisc468:~/a1$ ./task2
Success!
Timestamp: 1767757009
Date: 2026-01-07 03:36:49
Key: b923903ed2a09312ec15d364d9c45d1b
```

We can see that Alice's approach to the key generation is very exploitable because:

- Not information-theoretically secure: we broke it in just a few seconds! The amount of computational power needed is very low to crack the key.
- Failed under the known-plaintext attack (we were given 1 plaintext ciphertext pair)
- The enemy knows the system. If we knew Alice used the system time as a seed, we could break the code in no time.

2.

The strategy included:

√ Pre-compute the first encryption prior to the for loop iteration

This helps us save the ^ operations inside the for loop (lowers total iterations by n)

```c
// Known values
unsigned char plaintext[] = {
    0x25, 0x50, 0x44, 0x46, 0x2d, 0x31, 0x2e, 0x35,
    0x0a, 0x25, 0xbf, 0xf7, 0xa2, 0xfe, 0x0a, 0x33
};
unsigned char iv[] = {
    0x6f, 0x6d, 0x65, 0x67, 0x61, 0x6c, 0x75, 0x6c,
    0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37, 0x38
};
unsigned char ciphertext[] = {
    0xb6, 0x44, 0xd7, 0x96, 0x19, 0x44, 0x85, 0x4b,
    0x65, 0x5f, 0x40, 0xca, 0xad, 0xb3, 0x0e, 0xda
};

// Pre-computed XOR of Plaintext and IV for optimization
unsigned char input_data[16];
for(int i=0; i<16; i++) {
    input_data[i] = plaintext[i] ^ iv[i];
}
```

√ Calculate start and end time, and initialize variables and OverSSL object

```
/* Time window: 2026-01-06 23:45:49 back 2 hours
21ky11@LINUX3:~/468$ date -d "2026-01-06 23:45:49" +%s
1767761149

21ky11@LINUX3:~/468$ date -d "2026-01-06 21:45:49" +%s
1767753949
*/
time_t start_time = 1767753949;
time_t end_time = 1767761149;

unsigned char key[16];
unsigned char out_buf[16];
int out_len;
int found = 0;

// Initialize OpenSSL EVP
EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();
if (!ctx) {
    perror("EVP_CIPHER_CTX_new");
    return 1;
}
```

Use a for loop to iterate through all possible timestamps within the estimated range, and attempt to encrypt the plaintext ⊕ initialization vector to check if it matches the known output ciphertext.

```
for (time_t t = start_time; t <= end_time; t++) {
    // Generate key at that timestamp
    srand(t);
    for (int i = 0; i < 16; i++) {
        key[i] = rand() % 256;
    }

    // Use ECB mode on the XORed input to simulate one block of CBC
    EVP_EncryptInit_ex(ctx, EVP_aes_128_ecb(), NULL, key, NULL);
    EVP_CIPHER_CTX_set_padding(ctx, 0); // No padding for full block

    if (EVP_EncryptUpdate(ctx, out_buf, &out_len, input_data, 16) != 1) {
        continue;
    }
```

```
        // Check if encrypted output matches the known ciphertext
        if (memcmp(out_buf, ciphertext, 16) == 0) {
            printf("Success!\nTimestamp: %ld\n", t);
            char time_str[100];                     // Buffer for formatted date/time
            struct tm *tm_info = localtime(&t);
            strftime(time_str, sizeof(time_str), "%Y-%m-%d %H:%M:%S", tm_info);
            printf("Date: %s\n", time_str);         // Print human-readable date
            print_hex("Key", key, 16);
            found = 1;
            break;
        }
    }
```

Task 3: Generating Cryptographic Keys

Modified:

```
#include <stdio.h>
#include <stdlib.h>

#define LEN 32 // 256 bits

int main() {
    // Given code snippet
    unsigned char *key = (unsigned char *) malloc(sizeof(unsigned char)*LEN);
    FILE* random = fopen("/dev/urandom", "r");
    fread(key, sizeof(unsigned char)*LEN, 1, random);
    fclose(random);

    // Print the generated key in hex
    printf("Generated 256-bit key:\n");
    for (int i = 0; i < LEN; i++) {
        printf("%.2x", key[i]);
    }
    printf("\n");
    free(key);
    return 0;
}
```

```
seed@cisc468:~/a1$ ./task3
Generated 256-bit key:
c54120a11e58861423cf3f3e26a4e23ab6b8c29c9ff76e209cf012ccd5ee7cc2
seed@cisc468:~/a1$ ./task3
Generated 256-bit key:
374e0cf6e30ce5f56814ceb2e248916f0cddcbde8241764f713dadd335ab9bd5
```

Task 4: Examining the Kernel's Entropy Sources

```
seed@cisc468:~/a1$ grep 'add_.*_randomness' /proc/kallsyms
0000000000000000 t add_hwgenerator_randomness.cold
0000000000000000 T __pfx_add_device_randomness
0000000000000000 T add_device_randomness
0000000000000000 T __pfx_add_interrupt_randomness
0000000000000000 T add_interrupt_randomness
0000000000000000 T __pfx_add_vmfork_randomness
0000000000000000 T add_vmfork_randomness
0000000000000000 t __pfx_add_timer_randomness
0000000000000000 t add_timer_randomness
0000000000000000 T __pfx_add_input_randomness
0000000000000000 T add_input_randomness
0000000000000000 T __pfx_add_disk_randomness
0000000000000000 T add_disk_randomness
0000000000000000 T __pfx_add_hwgenerator_randomness
0000000000000000 T add_hwgenerator_randomness
0000000000000000 r __ksymtab_add_device_randomness
0000000000000000 r __ksymtab_add_disk_randomness
0000000000000000 r __ksymtab_add_hwgenerator_randomness
0000000000000000 r __ksymtab_add_input_randomness
0000000000000000 r __ksymtab_add_interrupt_randomness
0000000000000000 r __ksymtab_add_vmfork_randomness
0000000000000000 T __pfx_add_bootloader_randomness
0000000000000000 T add_bootloader_randomness
```

All of these add_*_randomness functions are designed to increase the amount of randomness/entropy when generating random numbers

- **[Discussed]** add_device_randomness:
  - Used to add data that is unique to the device but not necessarily random (like MAC addresses, serial numbers, or boot time) to the entropy pool to prevent identical boot states across different machines.
- **[Discussed]** add_interrupt_randomness: extract hardware interrupts like disk I/O, network activities
- **[Discussed]** add_disk_randomness: similar to add_blkdev_randomness, measuring finishing time of block device requests
- **[Discussed]** add_input_randomness: combination of add_keyboard_randomness add_mouse_randomness

- add_hwgenerator_randomness: Adds actual random numbers from a specialized hardware Random Number Generator (hwrng/TPM chip) directly into the kernel's entropy pool.
- add_timer_randomness: Extracts execution time of internal kernel timers (lower entropy because timers are relatively predictable)
- add_bootloader_randomness: Takes random seed data passed from the bootloader during system startup to initialize the random number generator before other hardware sources are active → allows more random elements when seed is provided.
- add_vmfork_randomness: for virtual machines; adds a unique identifier (like a "VM generation ID") to the random pool when a VM is cloned or forked to prevent two cloned VMs from generating identical random distinct sequences.

Upon running the piece of python code with add interrupt randomness() added as a kernel probe, we can see lots of lines like:

```
b'    kworker/u8:3-5969    [001] dNh.1  8250.076768: bpf_trace_printk:
add_interrupt_randomness: irq=237'
b''
b'          <idle>-0       [000] dNh.1  8250.079957: bpf_trace_printk:
add_interrupt_randomness: irq=237'
b''
b'          <idle>-0       [001] d.h.1  8250.080099: bpf_trace_printk:
add_interrupt_randomness: irq=26'
b''
b'           sshd-1931    [000] d.h.1  8250.080195: bpf_trace_printk:
add_interrupt_randomness: irq=25'
b''
b'           sshd-1931    [000] d.h.1  8250.080292: bpf_trace_printk:
add_interrupt_randomness: irq=25'
b''
b'           sudo-12068   [000] d.h.1  8250.080382: bpf_trace_printk:
add_interrupt_randomness: irq=237'
b''
b'           node-2067    [001] d.h.1  8250.080382: bpf_trace_printk:
add_interrupt_randomness: irq=237'
b''
```

We observe that:

- Linux entropy is collected continuously, not just at boot
- Interrupts are a major source of randomness
- Each IRQ means a different “device,” the most common one (irq=237) is our most active device, that likely triggers and collects the most amount of interrupts

Provide your code and also provide screenshots demonstrating that your code is working. The details can be found in task4.py and injector_key.c snippets are attached here

```
from bcc import BPF

# BPF program
bpf_text = """
#include <uapi/linux/ptrace.h>
#include <linux/input.h>

// Trace function for input_handle_event
int trace_input_handle_event(struct pt_regs *ctx, void *dev, unsigned int type, unsigned
int code, int value)
{
   bpf_trace_printk("input_event:    type=%u code=%u val=%d\\n", type, code, value);
   return 0;
}

// some code omitted

# Initialize BPF
b = BPF(text=bpf_text)

# Attach kprobe to input_handle_event
b.attach_kprobe(event="input_handle_event",
   fn_name="trace_input_handle_event")

print("Tracing add_input_randomness... Ctrl-C to stop.")
b.trace_print()
```

Explanation: We needed to add handle_input_event instead of add_input_randomness because in the particular version of Linux I'm on, the add_input_randomness is inlined within this function (the handle_input_event calls input_pass_values, which calls add_input_randomness, docs).

This is a difference between this and the previously provided snippet: add_interrupt_randomness() is a non-inlined, globally visible function, so we could kprobe to trace its usage directly.

Testing that this works

To actually simulate the keyboard and mouse movements, we need another script that can interact with the Linux machine we're operating on.

```
// include statements omitted

void inject_mouse(const char *device_path) {
   // Open the mouse device file (e.g., /dev/input/eventX)
   int fd = open(device_path, O_WRONLY);
   if (fd < 0) {
       printf("Failed to open mouse device: %s\n", device_path);
       return;
   }

   struct input_event ev;

   // Move the mouse 5 units to the right
   memset(&ev, 0, sizeof(ev));
   ev.type = EV_REL;
   ev.code = REL_X;
   ev.value = 5;
   if(write(fd, &ev, sizeof(ev)) < 0) perror("write rel");

   // Sync event (the actual signal to the system that the event is complete)
   memset(&ev, 0, sizeof(ev));
   ev.type = EV_SYN;
   ev.code = SYN_REPORT;
   ev.value = 0;
   if(write(fd, &ev, sizeof(ev)) < 0) perror("write syn");

   printf("Injected Mouse Event to %s\n", device_path);
   close(fd);
}

void inject_keyboard(const char *device_path) {
   int fd = open(device_path, O_WRONLY);
   if (fd < 0) {
       printf("Failed to open keyboard device: %s\n", device_path);
       return;
   }

   struct input_event ev;
```

```
    // Key press 'A'
    memset(&ev, 0, sizeof(ev));
    ev.type = EV_KEY;
    ev.code = 30; // KEY_A
    ev.value = 1; // Press
    if(write(fd, &ev, sizeof(ev)) < 0) perror("write key press");

    // Sync event (the actual signal to the system that the event is complete)
    memset(&ev, 0, sizeof(ev));
    ev.type = EV_SYN;
    ev.code = SYN_REPORT;
    ev.value = 0;
    if(write(fd, &ev, sizeof(ev)) < 0) perror("write syn");

    // Release 'A'
    memset(&ev, 0, sizeof(ev));
    ev.type = EV_KEY;
    ev.code = 30; // KEY_A
    ev.value = 0; // Release
    if(write(fd, &ev, sizeof(ev)) < 0) perror("write key release");

    // Sync event (the actual signal to the system that the event is complete)
    memset(&ev, 0, sizeof(ev));
    ev.type = EV_SYN;
    ev.code = SYN_REPORT;
    ev.value = 0;
    if(write(fd, &ev, sizeof(ev)) < 0) perror("write syn");

    printf("Injected Keyboard Event to %s\n", device_path);
    close(fd);
}

int main() {
    printf("Attempting to inject events...\n");
    inject_mouse("/dev/input/event0");
    inject_keyboard("/dev/input/event1");
    return 0;
}
```

injector.c executable Output:

```
seed@cisc468:~/a1$ sudo ./injector_key
Attempting to inject events...
Injected Mouse Event to /dev/input/event0
Injected Keyboard Event to /dev/input/event1
```

task4.py Output:

```
b'    injector_key-9870    [000] d...1  7966.606360: bpf_trace_printk: input_event:
type=0 code=0 val=0'
b''
b'    injector_key-9870    [000] d...1  7966.610836: bpf_trace_printk: input_event:
type=1 code=30 val=1'
b''
b'    injector_key-9870    [000] d...1  7966.610845: bpf_trace_printk: input_event:
type=0 code=0 val=0'
b''
b'    injector_key-9870    [000] d...1  7966.610847: bpf_trace_printk: input_event:
type=1 code=30 val=0'
b''
b'    injector_key-9870    [000] d...1  7966.610848: bpf_trace_printk: input_event:
type=0 code=0 val=0'
b''
```

Interpretation of the output
We can see that our injector simulated: 1 mouse movement, and 1 key-press (press and release).
Both were successfully captured in the log of task4.py
The 5 logs mean:

- Simulated mouse movement synced (type=0)
- Simulated keyboard press (type=1, val=1)
- Simulated keyboard press synced (type=0)
- Simulated keyboard release (type=1, val=0)
- Simulated keyboard release synced (type=0)

Observation:
I originally expected that the mouse movement simulation would appear in the logs (e.g., an input event of type=2, code=0, val=5) because I moved 5 pixels. However, it was not present, because I found out that on some Linux machines, injected mouse motion events (EV_REL, type=2) written directly to a real mouse's /dev/input/eventX are often filtered or ignored by the device driver, since the driver only trusts motion data coming from actual hardware interrupts. The simulated mouse movement syncing and appearing in the logs means that the tracking is correctly established, though.

Task 5: Breaking a Reused One-Time Pad

```
C1: 2a b4 c7 ec ef 01 dc 75 f9 2d fe c2 6b 0e 1b 1e f8 99
C2: 39 a9 c3 be ee 4f cd 70 e5 2d e7 de 38 04 03 18 e4 92
```

Index: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

C1 XOR C2: 13 1d 04 **52** 01 **4e** 11 05 1c 00 19 1c **53** 0a 18 06 1c 0b

We can determine possible spaces by XORing the two ciphertext strings, as:
$C1 \oplus C2 = (P1 \oplus K) \oplus (P2 \oplus K) = P1 \oplus P2$
This means that the 2 ciphertexts XORed together are the result of XORing 2 plaintexts.

1. We can see that index 9 at both C1 and C2 is '2d'.
Furthermore, **index 9** of C1 XOR C2 has character 00. This also means they must be the same letter (because the same letter XORed with the same letter gets you 00).
Both cases implies that, at index 9, the plaintexts P1 and P2 must contain the same character.

2. Now from the ASCII Table we know that space has the hex code of 0x20
This means if one of the plaintext has a space at that byte, the result of P1 XOR P2 at that byte will be in the range of 0x41 to 0x5A. The XOR range of two lowercase letters is in the range of 0x00 to 0x1b (character zones)

0x61 (lowercase a) XOR 0x20 (space) = 0110 0001 XOR 0010 0000 = 0100 0001 = 0x41 (uppercase A)	0x7a (lowercase z) XOR 0x20 (space) = 0111 1010 XOR 0010 0000 = 0101 1010 = 0x5A (uppercase Z)
0x61 (lowercase a) XOR 0x61 (lowercase a)) = 0x00 (NUL character)	0x61 (lowercase a) XOR 0x7a (lowercase z) = 0x1b (ESC character)

In other words, if any byte at P1 XOR P2 is an uppercase letter, one of the bytes should be a space.
Edge case: what if both positions are spaces → It would produce 0x00!

We can see that bytes at index 3, 5, and 12 for one of P1 or P2 are **definitely** locations of a space.

Additionally, 0x00 at index 9 **might** be a space for both P1 or P2.

3. Just like how we previously XORed 2 plaintext, we use the property
$A \oplus B = C \Rightarrow A = B \oplus C \Rightarrow B = A \oplus C$
We have identified that
0x52 is at index 3: $0x52 \oplus 0x20 = 0x72 = "r"$
0x4e is at index 5: $0x4e \oplus 0x20 = 0x6e = "n"$
0x53 is at index 12: $0x53 \oplus 0x20 = 0x73 = "s"$

Now we know the following about our 18-character-long plaintext:

- One of P1 or P2 at index 3 (4th letter) is an r
- One of P1 or P2 at index 5 (6th letter) is a n
- One of P1 or P2 at index 12 (13th letter) is an s
- P1 and P2 are the same letter/space at index 9 (10th letter)
- P1 XOR P2 = C1 XOR C2

4.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
T	H	E		E	N	E	M	Y				' '					
G	U	A	R	D		T	H	E				S					

Thought process:
I'm gonna begin with a more constrained search.
It does not matter that the space at index 3 is at P1 or P2.
Let's assume index 3 of P1 is a space, and index 5 of P2 is a space.

We run this script, guessing the 3-letter word being one of the most common ones in English.

```
# List of words
words = [
   "all", "and", "are", "but", "can", "for", "get", "had", "has",
   "her", "him", "his", "how", "not", "now", "one", "out", "she",
   "the", "two", "was", "who", "why", "you"
]

# Hex codes to XOR with
xor_keys = [0x13, 0x1D, 0x04]

# Iterate through each word
for word in words:
   result_chars = []
```

```
    for i, char in enumerate(word):
        # XOR character with the corresponding key (cycled)
        xor_value = ord(char) ^ xor_keys[i % len(xor_keys)]
        result_chars.append(chr(xor_value))
    # Join the characters into a string
    result_text = ''.join(result_chars)
    print(f"{word} -> {result_text}")
```

The best looking result is:

0	1	2	3	4	5
t	h	e		e	n
g	u	a	r	d	

Next, we expand the search:

- Assume that both indices 9 are spaces (we can backtrack if nothing promising shows up)
- That means indices 4-8 in P1 is a 5 letter word starting with ‘en’
- That means indices 6-8 in P2 is a 3 letter word

We run this script (words_alpha.txt is acquired using `wget https://raw.githubusercontent.com/dwyl/english-words/master/words_alpha.txt`)

```
# Convert to bytes (first 10 chars)
C1 = bytes.fromhex("2ab4c7ecef01dc75f92dfec26b0e1b1ef899")[:10]
C2 = bytes.fromhex("39a9c3beee4fcd70e52de7de38040318e492")[:10]

# XOR stream for first 10 chars
X = bytes(c1 ^ c2 for c1, c2 in zip(C1, C2))

# Load wordlists
WORDS_3 = set()
WORDS_5_EN = set()
with open("words_alpha.txt") as f:
    for w in f:
        w = w.strip().lower()
        if w.isalpha():
            if len(w) == 3:
                WORDS_3.add(w)
            elif len(w) == 5 and w.startswith("en"):
                WORDS_5_EN.add(w)
print(f"Loaded {len(WORDS_5_EN)} 5-letter 'en*' words")
print(f"Loaded {len(WORDS_3)} 3-letter words")
```

```python
# Search for valid plaintext pairs
for w1 in WORDS_5_EN:
    for w2 in WORDS_3:
        # Build candidate plaintexts
        P1 = "the " + w1 + " "
        P2 = "guard " + w2 + " "
        # Check XOR consistency
        valid = True
        for i in range(10):
            if ord(P1[i]) ^ ord(P2[i]) != X[i]:
                valid = False
                break
        if valid:
            print(f"P1: {P1} | P2: {P2}")
```

The result we get:

```
seed@cisc468:~/a1$ python3 task5_4.py
Loaded 61 5-letter 'en*' words
Loaded 2130 3-letter words

Found 4 candidates:

P1: the ental     | P2: guard edp
P1: the enemy     | P2: guard the
P1: the entry     | P2: guard ewe
P1: the enami     | P2: guard phu
```

Looks like the second output is very likely (given we're in a cryptography class)!

That means the following configurations remain

1. P1 = "`The enemy XX XXXXX`" P2 = "`Guard the XXXXXXXX`"
2. P1 = "`The enemy XXXXXXXX`" P2 = "`Guard the XX XXXXX`"

Option 1 looks more likely, in fact, the next 2 letter word seems like the word "is"!
Given "is" (0x6973), XORed with 0x191c from C1 XOR C2, we get "po"
Then, because the next character is a space, the 8 letter word in P2 might start with "pos"

Therefore, we conduct one last search using this script:

```python
# Convert to bytes
C1 = bytes.fromhex("2ab4c7ecef01dc75f92dfec26b0e1b1ef899")
C2 = bytes.fromhex("39a9c3beee4fcd70e52de7de38040318e492")
```

```python
# XOR stream
X = bytes(c1 ^ c2 for c1, c2 in zip(C1, C2))

# Load wordlists
WORDS_5 = set()
WORDS_8_POS = set()
with open("words_alpha.txt") as f:
    for w in f:
        w = w.strip().lower()
        if w.isalpha():
            if len(w) == 5:
                WORDS_5.add(w)
            elif len(w) == 8 and w.startswith("pos"):
                WORDS_8_POS.add(w)
print(f"Loaded {len(WORDS_5)} 5-letter words")
print(f"Loaded {len(WORDS_8_POS)} 8-letter words starting with 'pos'")

# Search for valid plaintext pairs
for w5 in WORDS_5:
    for w8 in WORDS_8_POS:
        # Build candidate plaintexts
        P1 = "the enemy is " + w5
        P2 = "guard the " + w8
        if len(P1) != 18 or len(P2) != 18:
            continue
        # Check XOR consistency
        valid = True
        for i in range(18):
            if ord(P1[i]) ^ ord(P2[i]) != X[i]:
                valid = False
                break
        if valid:
            print(f"P1: {P1} | P2: {P2}")
```

And voila, only one result remains.

```
seed@cisc468:~/a1$ python3 task5_5.py
Loaded 15921 5-letter words
Loaded 65 8-letter words starting with 'pos'
P1: the enemy is close | P2: guard the position
```